Unit- 1

Unit-1: Operating System Introduction — Topic Breakdown

Main Topic	Subtopics to Study	What to Focus for Exam
Introduction to Operating System	Definition of OS, Need of OS, Functions of OS	2–3 line definition + list of functions
Objectives of Operating System	Convenience, Efficiency, Ability to evolve	Short explanation of each objective
Evolution of Operating Systems	Early computer systems, Batch systems, Multiprogramming systems, Time-sharing systems	Chronological understanding
Types of Operating Systems	Batch OS, Multiprogramming OS, Time-sharing OS, Real-time OS, Distributed OS, Network OS	Definition + example
Operating System Services	Program execution, I/O operations, File system manipulation, Communication, Error detection	List with explanation
System Calls	Definition, Purpose of system calls	Concept explanation
Types of System Calls	Process control, File management, Device management, Information maintenance, Communication	Examples like fork(), exec()
Operating System Structure	Simple structure, Layered structure, Microkernel, Modular approach	Diagram based questions
Virtual Machines	Concept of virtualization, Benefits	Short explanation
Operating System Examples	Linux, Windows, UNIX, macOS	Use as examples in answers

---

1. Definition of Operating System (OS)

An **Operating System (OS)** is system software that acts as an **interface between the user and computer hardware**, managing hardware resources and providing services for application programs.

Simple definition for exam:

An operating system is a program that manages computer hardware resources and provides an environment for executing application programs.

Key Points

- Acts as **intermediary between user and hardware**
- Manages **CPU, memory, I/O devices, and files**
- Provides **services for program execution**

Basic Structure

User
↓
Application Programs
↓
Operating System
↓
Computer Hardware

2. Need of Operating System

An operating system is required to make the computer system **usable, efficient, and organized**.

Need	Explanation
Hardware Management	Controls CPU, memory, and input/output devices
Program Execution	Loads and runs application programs
Resource Allocation	Allocates CPU time, memory, and devices among processes
User Interface	Provides CLI or GUI for user interaction
File Management	Organizes and stores data in files and directories
Security	Protects data and controls access

Short exam statement

Without an operating system, users would need to control hardware directly, which would make computer operation complex and inefficient.

3. Functions of Operating System

The operating system performs several important functions to manage the computer system.

Function	Description
Process Management	Creates, schedules, and terminates processes
Memory Management	Allocates and deallocates memory for programs
File System Management	Manages files, directories, and storage
Device Management	Controls input/output devices
Security and Protection	Prevents unauthorized access
User Interface	Provides CLI or GUI to interact with system
Resource Allocation	Distributes system resources efficiently
Error Detection	Detects and handles system errors

Important line for exams

The primary function of an operating system is to manage system resources efficiently and provide a convenient environment for program execution.

Objectives of Operating System

The **objectives of an Operating System** describe the main goals that an OS tries to achieve while managing computer resources and providing services to users.

Main Objectives of Operating System

Objective	Explanation
Convenience	The OS makes the computer system easy to use by providing a user-friendly environment for executing programs.
Efficiency	It manages hardware resources such as CPU, memory, and I/O devices efficiently to improve system performance.
Resource Management	The OS allocates system resources among multiple users and processes in a controlled and fair manner.
Ability to Evolve	The OS should be designed so that new features, hardware, and system functions can be added easily without disturbing existing services.
Reliability	The OS ensures stable and reliable operation of the computer system.
Security and Protection	It protects system resources and data from unauthorized access or misuse.

- The objectives of an operating system are to make the computer system convenient to use, utilize hardware resources efficiently, and provide a secure and reliable environment for program execution.

Evolution of Operating Systems

The **evolution of operating systems** describes how operating systems developed over time to improve **efficiency, resource utilization, and user interaction**.

Stages in Evolution of Operating Systems

Stage	Explanation	Key Features
Serial Processing (No OS)	Early computers had no operating system. Programs were executed manually by users using punch cards.	Single program execution, manual setup
Batch Operating Systems	Jobs with similar requirements were grouped together and executed in batches without user interaction.	Job queues, reduced setup time
Multiprogramming Operating Systems	Multiple programs were loaded into memory at the same time to improve CPU utilization.	CPU switches between processes
Time-Sharing Systems	CPU time was divided among multiple users, allowing interactive use of the system.	Multi-user systems, quick response time
Real-Time Operating Systems	Systems designed to process data and provide responses within a strict time limit.	Used in embedded systems and control systems
Distributed Operating Systems	Multiple computers connected through a network work together and share resources.	Resource sharing, load balancing

Chronological Evolution

Serial Processing
↓
Batch Systems
↓
Multiprogramming Systems
↓
Time-Sharing Systems
↓
Real-Time Systems
↓
Distributed Systems

Key Idea for Exams : Operating systems evolved from simple manual processing systems to advanced systems capable of handling multiple users, processes, and distributed computing environments.

Types of Operating Systems

Operating systems are classified based on **how they manage processes, users, and resources**. Each type was developed to solve specific computing problems.

Batch Operating System

1. Definition : A **Batch Operating System** is an operating system in which **similar jobs are collected together and processed as a group (batch) without direct user interaction**.

In a batch operating system, jobs are grouped into batches and executed automatically one after another by the operating system.

2. Concept of Batch Processing

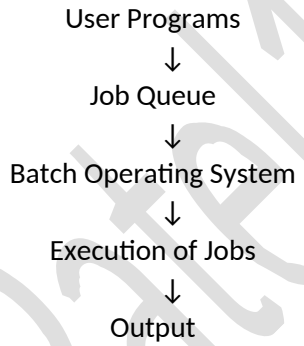
In early computer systems, users submitted programs using **punch cards or magnetic tapes**. The operating system collected many jobs and executed them **sequentially as a batch**.
Users did not interact with the system during execution.
The system processed jobs **one by one automatically**.
Example tasks processed in batches:

- Payroll processing
- Bank statement generation
- Billing systems
- Data processing jobs

3. Working of Batch Operating System
Steps involved in batch processing:

1. Users prepare programs and submit them to the system.
2. The system operator groups similar jobs into a batch.
3. Jobs are stored in a job queue.
4. The operating system executes jobs sequentially.
5. Results are produced after execution.

Basic Workflow



4. Components of Batch Operating System

Component	Description
Job Queue	Stores all submitted jobs waiting for execution
Batch Monitor	Manages execution of jobs in batches
Input Spooling	Collects jobs before execution
Output Spooling	Stores output until printing

5. Characteristics of Batch Operating System

Feature	Description
No user interaction	Users cannot interact during execution
Automatic job execution	OS executes jobs sequentially
Similar jobs grouped	Jobs with similar requirements are grouped
High throughput	Many jobs processed together
Long turnaround time	Users wait longer for results

6. Advantages of Batch Operating System

Advantage	Explanation
Efficient for large jobs	Suitable for processing large volumes of data
Reduced idle CPU time	CPU utilization improves
Automatic execution	No manual intervention during execution

Easy job scheduling	Similar jobs processed together

7. Disadvantages of Batch Operating System

Disadvantage	Explanation
No user interaction	Users cannot modify jobs during execution
Long waiting time	Output may take hours or days
Difficult debugging	Errors found after execution
Not suitable for small tasks	Small tasks must wait in queue

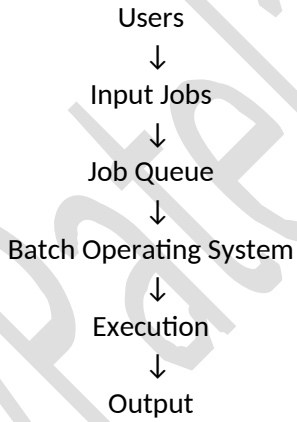
8. Real World Examples

Examples where batch systems are still used:

Application	Example
Banking systems	Daily transaction processing
Payroll systems	Salary calculation
Billing systems	Electricity bill generation
Data processing	Large database updates

9. Diagram for Exam

You can draw this simple diagram:



10. Short Conclusion

A batch operating system processes jobs in groups without user interaction. It was widely used in early computer systems for handling large volumes of data processing tasks efficiently.

Multiprogramming Operating System

1. Definition : A **Multiprogramming Operating System** is an operating system that allows **multiple programs to be loaded into memory and executed by the CPU alternately**.

In a multiprogramming operating system, several programs are kept in main memory at the same time, and the CPU switches between them to keep itself busy.

2. Concept of Multiprogramming

In a simple system, when a program performs **I/O operations**, the CPU remains idle.

Multiprogramming solves this problem.

When one program waits for I/O, the CPU executes **another program**.

This improves **CPU utilization and system efficiency**.

Example:

- Program A → waiting for disk input

- CPU switches to → Program B
- Program B executes until it needs I/O

3. Working of Multiprogramming OS

Steps of operation:

1. Multiple programs are loaded into main memory.
2. The operating system selects one program for execution.
3. If the program performs I/O, the CPU switches to another program.
4. This switching continues until all programs complete.

Basic Workflow

Multiple Programs in Memory
↓
CPU Scheduler
↓
Program Execution
↓
I/O Request
↓
CPU switches to another program

4. Requirements for Multiprogramming

Requirement	Description
Large Main Memory	To store multiple programs simultaneously
CPU Scheduling	Selects the next program for execution
Memory Management	Allocates memory to different programs
I/O Management	Handles multiple I/O requests

5. Characteristics of Multiprogramming OS

Feature	Description
Multiple programs in memory	More than one program stored in RAM
CPU switching	CPU alternates between programs
High CPU utilization	CPU rarely remains idle
Efficient resource usage	Better use of memory and devices

6. Advantages of Multiprogramming

Advantage	Explanation
Improved CPU utilization	CPU works continuously
Increased system throughput	More programs completed in less time
Efficient use of resources	Memory and devices used efficiently
Reduced idle time	CPU does not wait for I/O

7. Disadvantages of Multiprogramming

Disadvantage	Explanation
Complex OS design	Requires advanced memory and process management
Higher memory requirement	Multiple programs require more memory
Difficult scheduling	OS must manage multiple programs efficiently

8. Example Systems

Examples of systems that use multiprogramming:

System	Example
Mainframe systems	Early IBM systems
Modern operating systems	Linux, Windows, Unix

Modern OS combine **multiprogramming with multitasking**.

9. Diagram for Exam

You can draw this simple diagram:

Main Memory

Program A
Program B
Program C
Program D

↓
CPU
(Switches between programs)

10. Short Conclusion

A multiprogramming operating system increases CPU utilization by keeping multiple programs in memory and executing them alternately whenever one program waits for input/output operations.

Time-Sharing Operating System

1. Definition : A **Time-Sharing Operating System** is an operating system that allows **multiple users to share the CPU simultaneously by allocating a small time slice to each user or process**.
In a time-sharing system, the CPU switches rapidly between multiple users or processes, giving the impression that each user has a dedicated system.

2. Concept of Time Sharing

Time-sharing systems were developed to allow **interactive computing**.
The CPU divides its time into **small intervals called time slices (time quantum)**.
Each process or user gets a small portion of CPU time.
After the time slice expires, the CPU switches to the next process.
Because switching happens very quickly, users feel that the system responds **immediately**.
Example:

- User A running a program
- User B editing a file
- User C executing commands

The CPU serves all users **alternately in milliseconds**.

3. Working of Time-Sharing OS

Steps involved:

1. Multiple users connect to the system through terminals.
2. Several processes are loaded into memory.
3. CPU scheduler allocates a small time slice to each process.
4. When the time slice expires, CPU switches to the next process.
5. This continues until all tasks are completed.

Basic Workflow

User A → Process A
User B → Process B
User C → Process C
↓
CPU
(Time slice allocated to each process)

4. Characteristics of Time-Sharing OS

Feature	Description
Multi-user system	Many users can use the system simultaneously
Interactive computing	Users can interact directly with the system
Time slicing	CPU time divided into small units
Fast response time	Quick response for user commands
Process switching	CPU switches rapidly between processes

5. Advantages of Time-Sharing OS

Advantage	Explanation
Multiple users supported	Many users can work simultaneously
Quick response time	Immediate interaction with system
Efficient CPU usage	CPU continuously executes processes
Improved productivity	Users can run programs interactively

6. Disadvantages of Time-Sharing OS

Disadvantage	Explanation
Security issues	Multiple users may create security risks
System overhead	Frequent context switching consumes CPU time
High hardware requirement	Requires powerful CPU and memory
System failure impact	If the system crashes, all users are affected

7. Examples of Time-Sharing Systems

Operating System	Example
UNIX	Multi-user time-sharing OS
Linux	Supports multiple users
Multics	Early time-sharing system
Modern servers	Shared computing environments

8. Diagram for Exam
Process A → Time Slice 1
Process B → Time Slice 2
Process C → Time Slice 3
Process D → Time Slice 4

CPU switches continuously between processes

9. Short Conclusion
A time-sharing operating system allows multiple users to share system resources simultaneously by allocating small time slices of CPU time, providing interactive and efficient computing.

Real-Time Operating System (RTOS)

1. Definition : A **Real-Time Operating System (RTOS)** is an operating system designed to **process data and provide a response within a fixed time limit (deadline)**.

A real-time operating system ensures that critical tasks are completed within a specified time constraint.

The correctness of the system depends **not only on logical results but also on the time at which the results are produced**.

2. Concept of Real-Time Systems

In many applications, the system must respond **immediately to external events**.

If the system fails to respond within the required time, it may lead to **system failure or serious consequences**.

Example:

- Aircraft control systems
- Medical monitoring systems
- Industrial automation
- Traffic control systems

These systems require **deterministic and predictable response time**.

3. Characteristics of Real-Time Operating System

Feature	Description
Deterministic behavior	Tasks are completed within predictable time limits
Fast response time	Quick reaction to external events
High reliability	System must operate continuously without failure
Priority-based scheduling	Critical tasks get higher priority
Minimal latency	Very low delay in processing

4. Types of Real-Time Operating Systems

Type	Description	Example
Hard Real-Time System	Missing a deadline may cause system failure	Aircraft control system
Soft Real-Time System	Missing a deadline reduces performance but does not cause failure	Multimedia streaming

5. Working of RTOS

Steps involved:

1. External event occurs (sensor input, signal).
2. RTOS receives the event.
3. High-priority task is scheduled immediately.
4. System processes the task within the deadline.
5. Output is generated in real time.

Basic Workflow

External Event
↓
Interrupt Signal
↓
RTOS Scheduler
↓

Task Execution
↓
Immediate Response

6. Advantages of Real-Time Operating System

Advantage	Explanation
Fast response	Immediate reaction to external events
High reliability	Suitable for critical applications
Deterministic system behavior	Predictable execution time
Efficient resource management	Handles priority tasks effectively

7. Disadvantages of Real-Time Operating System

Disadvantage	Explanation
Complex design	Requires specialized scheduling algorithms
Expensive hardware	Often needs high-performance systems
Limited flexibility	Designed for specific tasks

8. Examples of Real-Time Operating Systems

RTOS	Application
VxWorks	Aerospace and defense
QNX	Automotive systems
RTLinux	Embedded systems
FreeRTOS	Microcontroller systems

9. Diagram for Exam

Sensor Input
↓
Real-Time Operating System
↓
Immediate Processing
↓
Output / Control Signal

10. Short Conclusion

A real-time operating system is designed for applications that require immediate and predictable responses within strict time constraints, making it suitable for critical systems such as aerospace, industrial control, and medical devices.

Distributed Operating System

1. Definition : A **Distributed Operating System (DOS)** is an operating system that manages a group of **independent computers connected through a network and makes them appear as a single system to users**.

A distributed operating system allows multiple computers to work together and share resources such as files, memory, and processors.

2. Concept of Distributed Systems

In a distributed system, several computers (nodes) are connected through a network.
Each computer has its **own processor and memory**, but they cooperate to perform tasks.

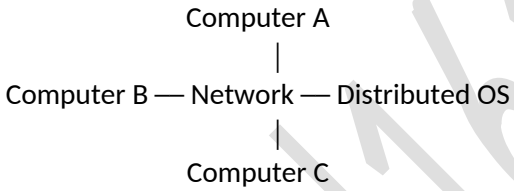
Users interact with the system **as if it were a single computer**, even though multiple machines are working together.
Example:

- Cloud computing systems
- Distributed databases
- Large-scale web services

3. Working of Distributed Operating System
Steps involved:

1. Multiple computers are connected through a network.
2. Each computer runs its own local operating system.
3. The distributed OS coordinates tasks among different machines.
4. Tasks and resources are shared across the network.
5. Results are returned to the user.

Basic Workflow



All systems cooperate to perform tasks.

4. Characteristics of Distributed OS

Feature	Description
Resource sharing	Files, printers, and memory can be shared across systems
Transparency	Users see the system as a single machine
Scalability	System can expand by adding more computers
Fault tolerance	System continues working even if one node fails
Load balancing	Workload distributed among multiple systems

5. Advantages of Distributed OS

Advantage	Explanation
High performance	Tasks distributed among multiple computers
Resource sharing	Hardware and software resources shared easily
Reliability	Failure of one node does not stop the entire system
Scalability	New computers can be added easily

6. Disadvantages of Distributed OS

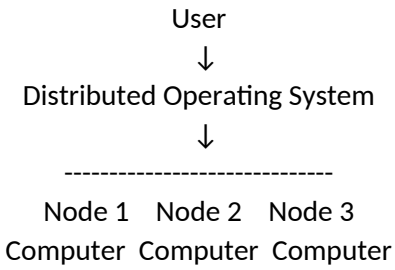
Disadvantage	Explanation
Complex system design	Difficult to design and manage
Network dependency	System depends on network performance
Security challenges	Data shared across network increases risks
Higher cost	Requires networking infrastructure

7. Examples of Distributed Systems

System	Application
Google Distributed System	Search and data processing
Hadoop Distributed System	Big data processing

Distributed databases	Data management across multiple servers

8. Diagram for Exam



9. Short Conclusion

A distributed operating system connects multiple computers through a network and manages them as a single system, enabling efficient resource sharing, scalability, and improved performance.

Network Operating System (NOS)

1. Definition: A **Network Operating System (NOS)** is an operating system that **manages and coordinates multiple computers connected through a network**, allowing them to communicate and share resources.

A network operating system provides services that allow computers to share files, printers, applications, and network resources over a network.

Unlike distributed systems, each computer **retains its own identity and operating system**.

2. Concept of Network Operating System

In a network operating system:

- Multiple computers are connected through a **local area network (LAN) or wide area network (WAN)**.
- Each computer runs its **own operating system**.
- Network services allow users to **access shared resources**.

Example:

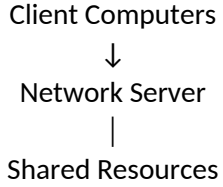
- Accessing a file stored on a server
- Printing through a network printer
- Sharing applications across systems

3. Working of Network Operating System

Steps involved:

1. Computers are connected through a network.
2. A server manages network resources.
3. Client computers send requests to the server.
4. The server provides access to files, printers, or applications.
5. Communication happens through network protocols.

Basic Workflow



(Files, Printers, Applications)

4. Characteristics of Network OS

Feature	Description
Resource sharing	Files, printers, and applications shared across network
Client-server architecture	Server manages network resources
Remote access	Users can access resources from remote systems
Security management	User authentication and access control
Network communication	Systems communicate using network protocols

5. Advantages of Network OS

Advantage	Explanation
Resource sharing	Hardware and software shared among computers
Centralized management	Server controls network resources
Easy data access	Users can access files from different locations
Scalability	New computers can be added easily

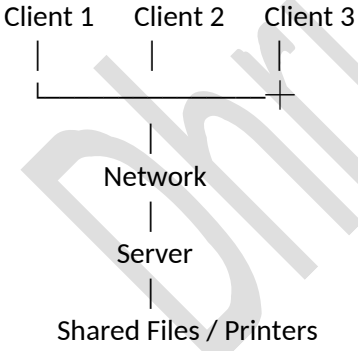
6. Disadvantages of Network OS

Disadvantage	Explanation
Server dependency	Failure of server affects the network
Security risks	Unauthorized access possible if not properly managed
Network congestion	Heavy network traffic may reduce performance
Setup cost	Requires networking infrastructure

7. Examples of Network Operating Systems

Network OS	Example
Windows Server	Enterprise networks
UNIX / Linux servers	Network services
Novell NetWare	Early network OS
macOS Server	Apple network environments

8. Diagram for Exam



9. Short Conclusion

A network operating system allows multiple computers to communicate and share resources over a network while maintaining their individual operating systems and identities.

Operating System Services

1. Definition: Operating System Services are the **functions provided by the operating system to users and application programs to make the system convenient and efficient to use.**

Operating system services provide an environment for program execution and manage system resources effectively.
These services help both **users and programs interact with the computer system.**

Main Operating System Services

Service	Description
Program Execution	The OS loads programs into memory and executes them.
I/O Operations	OS manages communication between programs and input/output devices such as keyboard, mouse, printer, and disk.
File System Manipulation	OS creates, deletes, reads, writes, and manages files and directories.
Communication	Allows processes to communicate with each other through shared memory or message passing.
Error Detection	Detects and handles errors in CPU, memory, or I/O devices.
Resource Allocation	Allocates CPU time, memory, and I/O devices among multiple processes.
Accounting	Tracks resource usage such as CPU time and memory used by users or programs.
Protection and Security	Prevents unauthorized access to system resources and data.

Explanation of Important Services

1. Program Execution

The operating system is responsible for **loading a program into memory and running it**.
Tasks performed:

- Loading program into memory
- Running the program
- Handling program termination

Example:
Running applications such as **browser, compiler, or editor**.

2. I/O Operations

Programs cannot directly control hardware devices.
The operating system provides **I/O services**.
Example devices:

- Keyboard
- Mouse
- Printer
- Disk drives

OS manages **device drivers and communication** with hardware.

3. File System Manipulation

The OS manages files stored in storage devices.
Operations include:

- Creating files
- Deleting files
- Reading files
- Writing files
- Managing directories

Example:
create file
read file
write file
delete file

4. Communication
Processes running in the system must **exchange data**.
Two common methods:

- **Shared Memory**
- **Message Passing**

Example:
Process A sending data to Process B.

5. Error Detection
The OS constantly monitors the system to detect errors.
Possible errors:

- Memory errors
- Device errors
- Hardware failures
- Software errors

The OS takes corrective actions to **maintain system stability**.

6. Resource Allocation
The OS allocates system resources among multiple processes.
Resources include:

- CPU
- Memory
- Disk space
- Input/output devices

The OS ensures **efficient utilization of resources**.

7. Accounting
The operating system records **resource usage information**.
Example:

- CPU time used by a process
- Memory usage
- Disk usage

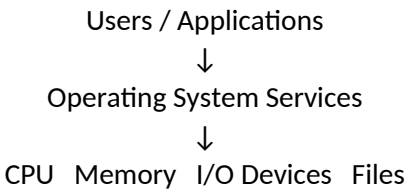
This helps in **system monitoring and performance analysis**.

8. Protection and Security
The OS protects system resources from unauthorized access.
Security functions include:

- User authentication
- Access control
- Data protection

Example:
Login systems with **username and password**.

Diagram for Exam



Short Conclusion

Operating system services provide essential functions such as program execution, I/O operations, file management, communication, resource allocation, and security to ensure efficient and reliable system operation.

System Calls

1. Definition: A **System Call** is a mechanism that allows a **user program to request services from the operating system kernel**.

System calls provide the interface between a user program and the operating system.

User programs cannot directly access hardware resources.

Instead, they request the operating system through **system calls**.

2. Concept of System Calls

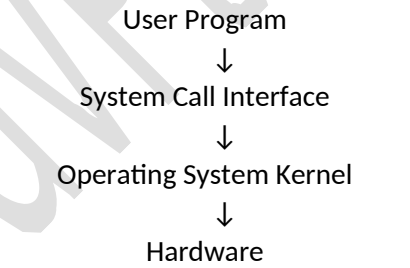
When a program needs a service such as:

- creating a file
- reading data from disk
- creating a process
- communicating with another process

it sends a **system call request to the operating system kernel**.

The kernel performs the requested operation and returns the result.

Basic Flow



3. Why System Calls are Needed

Reason	Explanation
Controlled access to hardware	Programs cannot directly access hardware
Security	Protects system resources
Resource management	OS manages CPU, memory, and devices
Standard interface	Provides uniform way for programs to request services

4. Types of System Calls

Type	Description	Example
Process Control	Manage processes	fork(), exit(), wait()
File Management	Create and manage files	open(), read(), write(), close()
Device Management	Control input/output devices	read(), write()

Information Maintenance	Get system information	getpid(), time()
Communication	Communication between processes	pipe(), shared memory

5. Examples of Common System Calls

System Call	Function
fork()	Creates a new process
exec()	Executes a new program
open()	Opens a file
read()	Reads data from a file
write()	Writes data to a file
close()	Closes a file
wait()	Waits for process completion

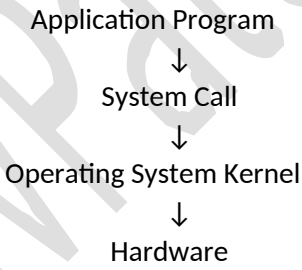
Example (Linux):

```
int fd = open("file.txt", O_RDONLY);
read(fd, buffer, size);
close(fd);
```

6. Characteristics of System Calls

Feature	Description
Interface between user and kernel	Connects user programs with OS services
Executed in kernel mode	Runs with high privileges
Used by application programs	Programs use them to access system resources
Essential for OS functionality	Enables safe interaction with hardware

7. Diagram for Exam



8. Short Conclusion

System calls provide a controlled interface through which user programs request services from the operating system kernel, enabling safe and efficient interaction with hardware and system resources.

Types of System Calls

System calls are classified based on the **type of service provided by the operating system to user programs**.

Main Types of System Calls

Type	Description	Examples
Process Control	System calls used to create, execute, and terminate processes.	fork(), exec(), exit(), wait()
File Management	Used for creating, reading, writing, and deleting files.	open(), read(), write(), close()
Device Management	Used to control and interact with hardware	read(), write(),

	devices.	ioctl()
Information Maintenance	Used to obtain or set system information such as time, process ID, and system data.	getpid(), time(), alarm()
Communication	Used for communication between processes.	pipe(), shmget(), msgsnd()
Protection	Used to control access permissions and security.	chmod(), chown()

Explanation of Each Type

1. Process Control System Calls

These system calls manage the **execution of processes**.

Functions include:

- Creating processes
- Terminating processes
- Loading programs into memory
- Waiting for process completion

Example:

fork()

exec()

exit()

wait()

Example usage:

Creating a new process in **Linux using fork()**.

2. File Management System Calls

These calls manage **files stored in storage devices**.

Operations include:

- Creating files
- Opening files
- Reading and writing files
- Deleting files

Example:

open()

read()

write()

close()

Example: reading a text file.

3. Device Management System Calls

These calls allow programs to **control hardware devices**.

Operations include:

- Requesting devices
- Releasing devices
- Reading from devices
- Writing to devices

Example devices:

- Printer
- Disk
- Keyboard

Example system calls:
read()
write()
ioctl()

4. Information Maintenance System Calls
These calls are used to **get or set system information**.
Information includes:

- Current time
- Process ID
- System status

Example:
getpid()
time()
uname()

5. Communication System Calls
These calls allow **data exchange between processes**.
Methods include:

- Message passing
- Shared memory
- Pipes

Example:
pipe()
msgsnd()
shmget()
Example: communication between two processes.

6. Protection System Calls
These system calls provide **security and access control**.
Functions include:

- Setting file permissions
- Changing ownership
- Managing access rights

Example:
chmod()
chown()

Diagram for Exam
User Program
↓
System Calls
↓

Process Control
File Management
Device Management
Information Maintenance
Communication

Protection

↓
Operating System Kernel

Short Conclusion
System calls are classified into categories such as process control, file management, device management, information maintenance, communication, and protection, depending on the type of services they provide to application programs.

Operating System Structure

1. Definition: Operating System Structure refers to the **internal design and organization of the operating system**, which determines how the OS components interact with each other and with the hardware.
It defines how different modules of the operating system are arranged and how they communicate to manage system resources efficiently.

Basic Structure of Operating System

Users
↓
Application Programs
↓
System Calls
↓
Operating System
↓
Hardware

The OS acts as an **intermediary between applications and hardware**.

Types of Operating System Structures

OS Structure	Description
Simple Structure	All OS components work together without clear separation.
Layered Structure	OS is divided into layers where each layer performs specific functions.
Microkernel Structure	Only essential services run in kernel; other services run in user space.
Modular Structure	OS is built using independent modules that can be added or removed.
Virtual Machine Structure	Hardware resources are shared to create multiple virtual machines.

1. Simple Structure
In this structure, the operating system is **not divided into modules or layers**.
All components are connected directly.
Example:

- Early UNIX systems
- MS-DOS

Characteristics

Feature	Description
No clear separation	OS components interact freely

Fast performance	Minimal overhead
Difficult to maintain	Changes may affect entire system

Diagram

Applications
↓
Operating System
(All components together)
↓
Hardware

2. Layered Structure

In the layered structure, the operating system is divided into **multiple layers**, where each layer performs a specific function.

Each layer communicates only with **adjacent layers**.

Characteristics

Feature	**Description**
Organized design	Clear separation of functions
Easy debugging	Errors isolated in layers
Modular design	Layers can be modified independently

Diagram

Layer 5 – User Interface
Layer 4 – File System
Layer 3 – Device Management
Layer 2 – Memory Management
Layer 1 – CPU Scheduling
Layer 0 – Hardware

3. Microkernel Structure

In a **microkernel architecture**, only the **essential components** of the OS are placed in the kernel.

Other services run in **user space**.

Core kernel services include:

- Process management
- Memory management
- Communication

Advantages

Advantage	**Explanation**
High reliability	Smaller kernel reduces errors
Better security	Services isolated
Easy to extend	New services added easily

Example Systems

- QNX
- MINIX

- Mach

Diagram

User Applications
↓
User-Level Services
↓
Microkernel
↓
Hardware

4. Modular Structure

In modular structure, the OS is divided into **independent modules**.
Each module performs a specific function and can interact with others.
Example:

- Linux kernel modules

Advantages

Advantage	Explanation
Flexible design	Modules can be added or removed
Easier maintenance	Modules updated independently
Efficient performance	Only required modules loaded

5. Virtual Machine Structure

In this structure, the OS creates **multiple virtual machines** that run separate operating systems on the same hardware.
Each virtual machine behaves like an independent computer.
Example:

- VMware
- VirtualBox
- Hyper-V

Diagram

Virtual Machine 1 → OS 1
Virtual Machine 2 → OS 2
Virtual Machine 3 → OS 3
↓
Virtual Machine Monitor
↓
Hardware

Short Conclusion

Operating system structure defines how OS components are organized and interact with each other. Different structures such as simple, layered, microkernel, modular, and virtual machine architectures are used to improve system efficiency, flexibility, and maintainability.

Virtual Machines (VM)

1. Definition: A **Virtual Machine (VM)** is a software-based simulation of a physical computer that allows multiple operating systems to run on a single hardware platform.
A virtual machine creates an isolated environment where an operating system and its applications can run as if they were on a separate physical computer.

Each virtual machine has its own:

- Operating system
- Memory
- CPU allocation
- Storage
- Applications

2. Concept of Virtual Machines

In traditional systems, one computer runs **one operating system**.
Virtualization allows a single physical computer to run **multiple operating systems simultaneously**.
This is done through a software layer called the **Virtual Machine Monitor (VMM) or Hypervisor**.
The hypervisor manages and allocates hardware resources among multiple virtual machines.
Example:
A single computer can run:

- Linux VM
- Windows VM
- macOS VM

simultaneously.

3. Structure of Virtual Machine System

Virtual Machine 1 → Guest OS
Virtual Machine 2 → Guest OS
Virtual Machine 3 → Guest OS
↓
Hypervisor (VMM)
↓
Hardware

4. Components of Virtual Machine

Component	Description
Host Machine	The physical computer running virtualization
Guest Operating System	OS installed inside a virtual machine
Hypervisor (VMM)	Software that manages virtual machines
Virtual Hardware	Simulated CPU, memory, disk, and network

5. Types of Virtual Machines

Type	Description
System Virtual Machine	Provides a complete operating system environment
Process Virtual Machine	Designed to run a single program

Example:

- **System VM:** VMware, VirtualBox
- **Process VM:** Java Virtual Machine (JVM)

6. Advantages of Virtual Machines

Advantage	Explanation
Efficient resource utilization	Multiple OS share the same hardware
Isolation	Problems in one VM do not affect others
Testing environment	Useful for testing software safely

Flexibility	Different operating systems can run simultaneously

7. Disadvantages of Virtual Machines

Disadvantage	Explanation
Performance overhead	Slightly slower than native systems
High resource usage	Requires more RAM and CPU
Complex management	Managing multiple VMs can be difficult

8. Examples of Virtual Machine Software

Software	Description
VMware	Popular virtualization platform
VirtualBox	Open-source VM software
Microsoft Hyper-V	Windows virtualization platform
KVM	Linux-based virtualization system

9. Example Use Cases

Application	Purpose
Software testing	Testing programs on different OS
Cloud computing	Running virtual servers
Security research	Running isolated environments
Development environments	Multiple OS for development

Short Conclusion

A virtual machine is a software-based computer that allows multiple operating systems to run on a single physical machine by sharing hardware resources through a hypervisor.

Operating System Examples

Operating systems are used to **manage computer hardware and provide services for application programs**. Different operating systems are designed for **personal computers, servers, mobile devices, and embedded systems**.

Common Examples of Operating Systems

Operating System	Developer	Description
Windows	Microsoft	One of the most widely used operating systems for personal computers.
Linux	Open-source community	A powerful open-source operating system used in servers, desktops, and embedded systems.
UNIX	AT&T Bell Labs	A multiuser and multitasking operating system widely used in servers and workstations.
macOS	Apple	Operating system used in Apple Macintosh computers.
Android	Google	A mobile operating system based on the Linux kernel used in smartphones and tablets.
iOS	Apple	Mobile operating system used in iPhones and iPads.

Brief Explanation of Major Operating Systems

1. Windows: Windows is a **graphical user interface (GUI) based operating system** developed by Microsoft.

Features:

- User-friendly interface
- Wide hardware support
- Large software ecosystem

Examples:

- Windows 10
- Windows 11

2. Linux: Linux is an **open-source operating system** based on the Unix architecture.

Features:

- Free and open source
- High security and stability
- Widely used in servers and cloud computing

Examples of Linux distributions:

- Ubuntu
- Fedora
- Debian
- Kali Linux

3. UNIX: UNIX is a **multiuser and multitasking operating system** developed at AT&T Bell Labs.

Features:

- Powerful command-line interface
- High stability
- Used mainly in servers and workstations

Examples:

- Solaris
- AIX
- HP-UX

4. macOS: macOS is the operating system developed by Apple for **Mac computers**.

Features:

- UNIX-based system
- Graphical interface
- High security and stability

5. Android: Android is a **Linux-based mobile operating system** developed by Google.

Features:

- Used in smartphones and tablets
- Open-source platform
- Large application ecosystem

6. iOS: iOS is Apple's **mobile operating system for iPhones and iPads**.

Features:

- Optimized for Apple hardware
- High security
- Smooth user experience

Classification of OS Examples

Category	Examples
Desktop OS	Windows, macOS, Linux

Server OS	Linux, UNIX, Windows Server
Mobile OS	Android, iOS
Embedded OS	RTOS, FreeRTOS

Short Conclusion

Examples of operating systems include Windows, Linux, UNIX, macOS, Android, and iOS, each designed for different computing environments such as personal computers, servers, and mobile devices.

Computer System Structure

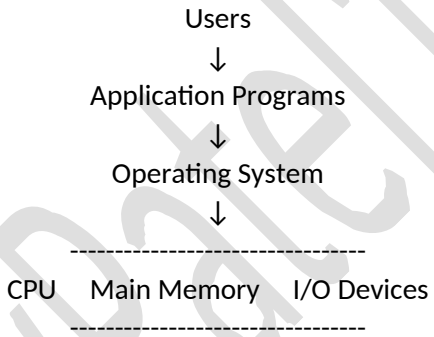
1. Definition: Computer System Structure describes **how the main hardware components of a computer are organized and how they interact with the operating system to execute programs**.

It explains the relationship between:

- CPU
- Main Memory
- Input/Output Devices
- System Bus
- Operating System

The operating system controls and coordinates all these components.

2. Basic Structure of a Computer System



Explanation:

Component	Role
CPU	Executes program instructions
Main Memory	Stores programs and data currently in use
I/O Devices	Handle input and output operations
Operating System	Manages all hardware resources

3. Main Components of Computer System

1. CPU (Central Processing Unit): The CPU is the **brain of the computer** that performs all processing tasks.

Main parts of CPU:

Part	Function
Control Unit	Controls execution of instructions
ALU	Performs arithmetic and logical operations
Registers	Small storage locations inside CPU

Functions of CPU:

- Fetch instructions from memory
- Decode instructions
- Execute instructions

2. Main Memory (RAM)

Main memory stores **programs and data currently being used by the CPU**.

Characteristics:

Feature	Description
Temporary storage	Data lost when power off
Fast access	Faster than secondary storage
Stores running programs	Required for execution

Example:When you open a program like a browser, it is loaded from disk into **RAM**.

3. Input / Output Devices: I/O devices allow communication between the user and the computer.

Examples:

Input Devices	Output Devices
Keyboard	Monitor
Mouse	Printer
Scanner	Speaker

The operating system manages these devices using **device drivers**.

4. Secondary Storage: Secondary storage stores data permanently.

Examples:

- Hard disk
- SSD
- USB drive
- CD/DVD

Characteristics:

Feature	Description
Permanent storage	Data remains after shutdown
Large capacity	Stores files and programs
Slower than RAM	Used for long-term storage

Programs are stored on disk and loaded into **main memory for execution**.

5. System Bus

A **system bus** connects CPU, memory, and I/O devices.

It transfers data between components.

Types of buses:

Bus Type	Function
Data Bus	Transfers actual data
Address Bus	Transfers memory addresses
Control Bus	Transfers control signals

Example flow:

CPU → Address Bus → Memory location

Memory → Data Bus → CPU

4. Working of Computer System

Step-by-step operation:

1. User runs a program.
2. Operating system loads the program from disk to RAM.
3. CPU fetches instructions from RAM.
4. CPU executes instructions.

5. I/O devices handle input and output operations.
6. OS manages memory, CPU scheduling, and device access.

5. Role of Operating System in Computer System

The operating system acts as **manager of the entire computer system**.

Main responsibilities:

Task	Explanation
CPU Management	Decides which program runs
Memory Management	Allocates memory to programs
Device Management	Controls I/O devices
File Management	Organizes files and storage
Security	Protects system resources

Without an OS, users would need to control hardware directly.

Interaction of Operating System with Hardware Architecture

1. Definition : Interaction of OS with Hardware Architecture refers to **how the operating system communicates with and controls hardware components such as CPU, memory, and I/O devices to execute programs efficiently**.

Application programs cannot access hardware directly.

They must request services through the **Operating System**.

2. Basic Interaction Flow

The operating system acts as a **middle layer** between applications and hardware.

User
↓
Application Program
↓
System Calls
↓
Operating System (Kernel)
↓
Hardware

(CPU, Memory, I/O Devices)

Explanation:

Layer	Role
User	Uses applications
Application	Sends requests
System Calls	Interface to OS
Operating System	Controls hardware
Hardware	Executes instructions

3. Interaction with CPU

The operating system manages the CPU to ensure efficient execution of processes.

OS responsibilities with CPU

Function	Description
Process Scheduling	Decides which process runs on CPU
Context Switching	Switches CPU between processes
Interrupt Handling	Responds to hardware signals
CPU Protection	Prevents illegal instructions

Example:
When multiple programs run:
Program A → CPU
Program B → waiting
Program C → waiting
The OS scheduler decides the execution order.

4. Interaction with Memory
The operating system manages **main memory (RAM)**.
OS responsibilities

Function	Description
Memory Allocation	Assign memory to programs
Memory Protection	Prevent programs from accessing others' memory
Address Translation	Convert logical address to physical address
Memory Deallocation	Free memory when program ends

Example:
RAM

Program A
Program B
Program C

OS ensures programs do not overwrite each other's memory.

5. Interaction with I/O Devices
Programs cannot directly control devices such as:

- keyboard
- printer
- disk
- network

The OS manages these devices through **device drivers**.
OS responsibilities

Function	Description
Device Management	Controls device access
Buffering	Temporarily stores data
Interrupt Handling	Handles device signals
Device Scheduling	Organizes device usage

Example: Application → OS → Printer Driver → Printer

6. Interrupt Mechanism: Hardware devices communicate with the OS using **interrupts**.
An interrupt is a signal sent to the CPU to request attention.
Example:

1. Disk finishes reading data.
2. Disk controller sends interrupt.
3. CPU stops current task.
4. OS handles the interrupt.

Basic flow:

Device

↓
Interrupt Signal
↓
CPU
↓
Operating System
↓
Service Routine

7. Dual Mode Operation

Operating systems use **two modes** to protect hardware.

Mode	Description
User Mode	Application programs run here
Kernel Mode	OS executes privileged instructions

Example:

User Mode
↓
Application running
↓
System Call
↓
Kernel Mode

Operating system controls hardware
This prevents programs from damaging the system.

8. Hardware Components Managed by OS

Hardware	OS Role
CPU	Scheduling and execution
Memory	Allocation and protection
I/O Devices	Device management
Disk Storage	File system management
Network	Communication management

9. Example of Complete Interaction

When a user opens a file:

User
↓
Text Editor
↓
System Call (open file)
↓
Operating System
↓
Disk Controller
↓
Hard Disk
↓
File loaded into memory

↓

CPU processes file

The OS coordinates all hardware components.

10. Simple Diagram for Exam

Draw this in exam:

Applications

↓

System Calls

↓

Operating System

↓

CPU | Memory | I/O Devices | Disk

---

11. Short Exam Answer (3–4 Marks)

Interaction of operating system with hardware architecture refers to the way the operating system communicates with and controls hardware components such as CPU, memory, and I/O devices. Applications request services through system calls, and the OS manages hardware resources using scheduling, memory management, device drivers, and interrupt handling to ensure efficient and secure system operation.

Multitasking Operating System

1. Definition: A **Multitasking Operating System** is an operating system that allows **multiple programs (tasks) to run apparently at the same time by sharing the CPU among them**.

The operating system rapidly switches the CPU between tasks, giving the illusion that all programs are running simultaneously.

Short exam definition:

A multitasking operating system allows multiple programs to execute concurrently by sharing CPU time among them.

2. Concept of Multitasking

In a single-task system, only **one program runs at a time**.

In a multitasking system, the CPU executes **many programs by switching between them quickly**.

Example:

A user can run:

- Web browser
- Music player
- Code editor
- File download

All at the same time.

Example flow:

Task A → CPU

Task B → waiting

Task C → waiting

CPU switches between tasks rapidly.

Because switching happens in milliseconds, users feel tasks are running simultaneously.

3. Working of Multitasking OS

Steps involved:

1. Multiple programs are loaded into memory.
2. The OS scheduler selects a task.
3. The CPU executes that task for a short time.
4. After the time slice expires, the OS switches to another task.
5. This continues until all tasks finish.

Basic workflow:

Multiple Programs
↓
CPU Scheduler
↓
CPU executes Task 1
↓
Time slice expires
↓
CPU switches to Task 2
↓
Process continues

This switching process is called **context switching**.

4. Types of Multitasking

1. Preemptive Multitasking: The operating system **automatically interrupts a running task** and switches to another task.

Characteristics:

- Controlled by OS
- Uses time slices
- More efficient

Examples:

- Windows
- Linux
- macOS

2. Cooperative Multitasking: A program continues running until it **voluntarily gives control back to the OS**.

Characteristics:

- Tasks decide when to release CPU
- Less efficient
- Rarely used today

Example:

- Early versions of Windows
- Early Mac OS

5. Characteristics of Multitasking OS

Feature	Description
Concurrent execution	Multiple programs run together
CPU sharing	CPU time divided among tasks

Fast context switching	CPU switches rapidly
Efficient resource usage	CPU rarely remains idle

6. Advantages of Multitasking OS

Advantage	Explanation
Improved productivity	Users run many programs simultaneously
Better CPU utilization	CPU rarely idle
Efficient resource usage	Memory and devices shared
User convenience	Smooth multitasking experience

Example: You can **write code while downloading files and listening to music**.

7. Disadvantages of Multitasking OS

Disadvantage	Explanation
High memory usage	Multiple programs require RAM
Complex OS design	Scheduling becomes difficult
System slowdown	Too many tasks reduce performance
Context switching overhead	CPU time used for switching tasks

8. Examples of Multitasking Operating Systems

Operating System	Description
Windows	Supports multiple applications
Linux	Multiuser multitasking system
macOS	UNIX-based multitasking OS
Android	Mobile multitasking system
iOS	Supports multiple background apps

9. Simple Diagram for Exam

You can draw this:
Program A → Time Slice 1
Program B → Time Slice 2
Program C → Time Slice 3
Program D → Time Slice 4
CPU switches rapidly between programs

10. Difference Between Multiprogramming and Multitasking

Feature	Multiprogramming	Multitasking
Concept	Multiple programs in memory	Multiple tasks run simultaneously
User interaction	Limited	Interactive
CPU switching	When program waits for I/O	After fixed time slice
Purpose	Improve CPU utilization	Improve user experience

11. Short Conclusion

A multitasking operating system allows multiple programs to run concurrently by sharing CPU time among them. It improves system efficiency, user productivity, and resource utilization by rapidly switching the CPU between tasks.

Parallel Operating System

1. Definition: A **Parallel Operating System** is an operating system designed to support **parallel processing**, where multiple processors work together to execute tasks simultaneously.

In a parallel OS, **several CPUs or cores cooperate to solve a problem faster by dividing the work among them**.
Short exam definition: A parallel operating system manages multiple processors that work simultaneously to execute programs and improve system performance.

2. Concept of Parallel Processing
In a traditional system: One CPU → executes one task at a time
In a parallel system: Multiple CPUs → execute different parts of a task simultaneously
Example: If a program requires heavy computation, the OS divides it into smaller tasks and distributes them across processors.
Example flow:
Task
↓
Divide into sub-tasks
↓
CPU1 → Subtask 1
CPU2 → Subtask 2
CPU3 → Subtask 3
CPU4 → Subtask 4
All processors work at the same time.

3. Purpose of Parallel Operating Systems
Parallel OS are designed to:

Purpose	Explanation
Increase performance	Multiple processors execute tasks faster
Improve reliability	System continues even if one processor fails
Handle complex computations	Suitable for scientific calculations
Efficient resource usage	Multiple CPUs used effectively

4. Types of Parallel Systems
1. Symmetric Multiprocessing (SMP)
In SMP systems:
- All processors share the same memory
- Each processor runs the same OS
- Work is distributed among CPUs

Example:
Shared Memory
|
CPU1 CPU2 CPU3 CPU4
Example systems:
- Linux SMP
- Windows multiprocessor systems

2. Asymmetric Multiprocessing (AMP)
In AMP systems:
- One processor controls the system
- Other processors perform specific tasks

Structure:

Master CPU → controls system
Slave CPUs → execute assigned tasks
Example: Embedded systems and specialized hardware systems.

5. Characteristics of Parallel Operating Systems

Feature	Description
Multiple processors	Uses more than one CPU
Parallel execution	Tasks run simultaneously
Shared resources	Processors share memory and devices
High performance	Faster execution of programs

6. Advantages of Parallel OS

Advantage	Explanation
High processing speed	Multiple CPUs work together
Better resource utilization	All processors used
Improved reliability	System continues if one processor fails
Handles complex tasks	Suitable for scientific computing

Example uses:

- Weather simulation
- Scientific research
- Artificial intelligence
- Large-scale data processing

7. Disadvantages of Parallel OS

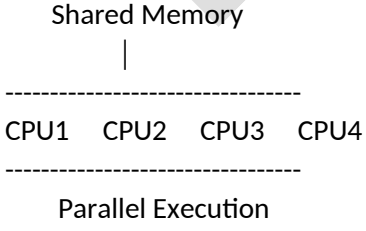
Disadvantage	Explanation
Complex design	Difficult to design and manage
Synchronization problems	Processors must coordinate
High cost	Requires multiple processors
Programming difficulty	Parallel programming is complex

8. Examples of Parallel Operating Systems

System	Example
Linux	Supports multiprocessor systems
Windows Server	Parallel processing support
Solaris	Multi-CPU systems
AIX	High-performance computing systems

9. Diagram for Exam

You can draw this simple diagram:



10. Short Conclusion

A parallel operating system allows multiple processors to execute tasks simultaneously, improving system performance, reliability, and efficiency. It is widely used in high-performance computing systems and large-scale scientific applications.

OS as Resource Manager

Definition : An **Operating System (OS) acts as a Resource Manager** because it manages and allocates the computer system's hardware resources among multiple users and programs efficiently.
Resources include:

- **CPU**
- **Main memory**
- **Input/Output devices**
- **Storage devices**
- **Network resources**

The OS decides **which process gets which resource and for how long**.

1. Why OS is Called a Resource Manager
A computer has **limited hardware resources**, but many programs may run simultaneously.
The operating system ensures these resources are used **efficiently, fairly, and safely**.
Main responsibilities:

Resource	OS Responsibility
CPU	Scheduling processes
Memory	Allocating RAM to programs
I/O Devices	Managing device access
Files	Organizing storage
Network	Managing communication

2. Resource Management Functions
1. CPU Management
The OS decides which process runs on the CPU.
Tasks:

- Process scheduling
- Context switching
- Process priority handling

Example:
If three programs run simultaneously, the OS schedules CPU time among them.

2. Memory Management
The OS allocates memory to different programs.
Functions:

- Memory allocation
- Memory deallocation
- Memory protection

Example:

Program	Memory Allocation
Program A	200 MB
Program B	150 MB
Program C	100 MB

The OS ensures programs **do not overwrite each other's memory**.

3. Device Management

The OS controls hardware devices through **device drivers**.

Examples:

- Printer
- Keyboard
- Disk
- Network card

Example: If two programs want to print, the OS **queues print jobs**.

4. File Management: The OS manages files stored in storage devices.

Operations:

- Create files
- Delete files
- Read/write files
- Manage directories

Example: Saving a document in a folder.

3. Goals of Resource Management

Goal	Explanation
Efficiency	Maximize hardware utilization
Fairness	Equal opportunity for processes
Protection	Prevent unauthorized access
Deadlock avoidance	Prevent system blocking
Performance	Improve response time

4. Simple Diagram for Exam

Users / Programs

↓

Operating System (Resource Manager)

↓

CPU | Memory | I/O Devices | Disk

The OS sits between **programs and hardware** and controls resource usage.

5. Example Scenario

Suppose three programs are running:

- Web Browser
- Music Player
- Code Editor

The OS will:

Resource	Action
CPU	Share CPU time among programs
Memory	Allocate RAM
Disk	Read/write files
Devices	Manage keyboard, screen, speaker

Thus, the OS manages all resources **simultaneously**.

Monolithic Kernel vs Microkernel

1. Monolithic Kernel

Definition : A **Monolithic Kernel** is an operating system architecture where **all OS services run inside the kernel space**.

These services include:

- Process management
- Memory management
- File system
- Device drivers
- System calls

All components work together in a **single large kernel program**.

Structure

Applications
↓
System Call Interface
↓

Monolithic Kernel
(Process, Memory, File System,
Device Drivers, Networking)

↓
Hardware

Characteristics

Feature	Description
Large kernel	Many services inside kernel
High performance	Direct communication between components
Less modular	Hard to modify individual parts
Lower security	A bug can crash the whole system

Advantages

- Fast execution
- Efficient communication between OS components
- High performance

Disadvantages

- Difficult to maintain
- Large kernel size
- Failure in one module can crash the entire system

Examples

- **Linux**
- **UNIX**
- **MS-DOS**

2. Microkernel

Definition: A **Microkernel** architecture keeps only **essential functions inside the kernel**, while other services run in **user space**.

Kernel contains only:

- Process scheduling
- Memory management
- Interprocess communication (IPC)

Other services run outside the kernel.

Structure

Applications
↓
User Space Services
(File system, Drivers, Network)
↓
Microkernel
(Process scheduling,
Memory management, IPC)
↓
Hardware

Characteristics

Feature	Description
Small kernel	Minimal core functions
High modularity	Services separated
Better security	Fault isolation
More communication overhead	Slower than monolithic

Advantages

- Higher security
- Easier to maintain
- More reliable
- Fault isolation

Disadvantages

- Slightly slower performance
- More communication overhead between modules

Examples

- **MINIX**
- **QNX**
- **Mach**

3. Monolithic vs Microkernel (Comparison)

Feature	Monolithic Kernel	Microkernel
Kernel size	Large	Small
Services location	Inside kernel	Outside kernel
Performance	Faster	Slightly slower
Security	Lower	Higher
Modularity	Low	High
Maintenance	Difficult	Easier

Communication	Direct	Message passing

4. Short 4–5 Mark Exam Answer

A **Monolithic Kernel** is an operating system architecture in which all system services such as memory management, device drivers, and file systems run inside the kernel space. It provides high performance but is difficult to maintain.

A **Microkernel** architecture includes only essential functions like process management, memory management, and communication inside the kernel. Other services run in user space, which improves security and modularity but may reduce performance.